

Sortierverfahren für hybride photonische Prozessoren

Bharadwaj Srikumar | Forschungsstand für Prof. Blunck | 5. Oktober 2026

Ziel: Untersuchen, unter welchen Korrektheits- und Ressourcenanforderungen sich Bitonic- und Rangsortierung sinnvoll auf hybride photonische Prozessoren abbilden lassen. Q.ANT dient als Referenz, soweit Quellen die Fähigkeiten belegen. Ein Energie- oder Geschwindigkeitsvorteil wird nicht vorausgesetzt.

1. Historische Modelle und Vergleichsrahmen

Modell	Beitrag zur Einordnung
Thompson (1979)	VLSI-Fläche und Zeit berücksichtigen Layout und Kommunikation. Fläche ist weder logischer Speicher noch die Zahl der Rechenpositionen.
Caulfield (1991)	Raum, Zeit und Fan-in gemeinsam betrachten. Wenige parallele Stufen können viele skalare Operationen enthalten.
Li & Monticone (2025)	Physische Ausdehnung und Kommunikation begrenzen lineare optische Operatoren. Diese Grenzen sind keine vollständige Sortierkomplexität.

C = (T, S, H, D) bleibt erhalten: T zählt die gesamte sequenzielle Arbeit einer Sortierung. S zählt gleichzeitig belegte logische Wörter einer Charge. H zählt installierte Verarbeitungspositionen. D zählt optische Stufen zwischen Regenerationen. T ist RAM-ähnlich; H und D bleiben architekturbezogen.

2. Komplexitäten der fünf Quellenarchitekturen

Architektur	T	S	H	D
Stirk & Athale: Pipeline	$\Theta(N \log^2 N)$	$O(N)$	$O(N \log^2 N)$	$O(\log^2 N)$ passiv; $O(1)$ restaurierend
Beyette: Rezirkulation	$\Theta(N \log^2 N)$	$O(N)$	$O(N)$	$O(1)$
Desmulliez: Perfect-Shuffle	$\Theta(N \log^2 N)$	$O(N)$	$O(N)$	$O(1)$
Louri: Rang / B&C;	$\Theta(N^2)$	$O(N^2)$	$O(N^2)$	$O(1)$
Mizutani: Netzwerk-Radix	$\Theta(N)^*$	$O(N)$	$O(1)^*$	$O(1)$

Bitonic hat $K = m(m+1)/2$ Stufen mit $m = \log_2 N$ und je $N/2$ Vergleichen. Rang verwendet $N(N-1)/2$ Paare. Louri's $O(1)$ -Angabe bezeichnet parallele optische Stufen, während T hier die gesamte skalare Arbeit erfasst.

* Radix: feste Wortbreite, Radix und Zahl der Verarbeitungsströme. Allgemein $T = \Theta(q(N+R))$; keine lineare Sortierung beliebiger reeller Zahlen. S beschreibt eine Charge, H keine Speicherfläche. Diese historischen H- und D-Werte werden nicht auf Q.ANT übertragen.

Thompson: doi:10.1145/800135.804401 · Caulfield: doi:10.1007/BF01937172 · Li & Monticone: doi:10.1038/s41467-025-63453-8
Stirk & Athale: doi:10.1364/AO.27.001721 · Beyette: doi:10.1364/AO.33.008164 · Desmulliez: doi:10.1364/AO.34.005077 · Louri: doi:10.1364/AO.34.003087 · Mizutani: doi:10.1364/JOCN.530695

Mapping und geprüfte Hardwaregrenze

3. Bisherige gemeinsame Vergleichsbasis

Gültige Schlüssel: 0-15 oder 0-255. Bei $b \in \{4,8\}$ gilt $L = 2^b$, $\hat{x} = x/L$ und $\Delta_{\text{in}} = \Delta_{\text{out}} = \Delta = 1/L$. Schlüssel-, Eingangs- und Ausgangsbreite bleiben getrennte Einstellungen.

$$d = \hat{a} - \hat{b} = [1 \ -1] [\hat{a} \ \hat{b}]^T; \varepsilon \sim N(0, (\eta\Delta)^2)$$

$$\tilde{d} = \Delta \cdot \text{clip}(\text{round}_{\text{even}}((d+\varepsilon)/\Delta), -L/2, L/2-1)$$

$\eta = \sigma/\Delta$ misst die Rauschstandardabweichung in Ausgangsschritten. Es bedeutet keine 5-25 % der Signalstärke. Das Modell rundet zur nächsten Zahl mit Halbwerten zur geraden Zahl. Sättigung erhält ohne Rauschen das Vorzeichen, aber nicht jeden Differenzbetrag.

Mapping	Differenzbildung	Elektronische Schritte
Bitonic nach Stirk & Athale	N/2 Paare je abhängiger Stufe	Vorzeichen und Originalindex auswerten, Originaldatensätze tauschen, nächste Stufe.
Rang nach Louri	Jedes ungeordnete Paar einmal	Siege als Ränge zählen, Ränge auf Eindeutigkeit prüfen, Originaldatensätze platzieren.

Die Rauschsimulation verwendet NumPy. Der separate SDK-Kontrolllauf ruft die lineare API auf dem offiziellen CPU-Backend auf. Beide sind funktionale Adaptionen, keine Reproduktionen der optischen Schaltung.

4. Q.ANT-Befund und mathematische Konsequenz

Operation im SDK 2.3.1	Geprüfter Befund
MVM und $y = v \cdot \text{tcos}(u)$	Eigene NPU-Treiberpfade. tcos ist als periodisch und kosinusähnlich dokumentiert.
ReLU: $r = \max(d,0)$	Host-CPU-Schleife in <code>src/non_linear.rs</code> ; kein Aufruf des NPU-Treibers.
Vollständiges optisches C&E / Kaskade	In der geprüften öffentlichen Schnittstelle nicht nachgewiesen.

Für genaue Werte gilt: $d = a-b$; $r = \text{ReLU}(d)$; $\min(a,b) = a-r$; $\max(a,b) = b+r$. Bei (10,9) ist $d=r=1$ und das Ergebnis (9,10). Der neue CPU-Check bestätigt diese Identität für alle 256 bzw. 65.536 Schlüsselpaare, ohne Zusatzrauschen und ohne Ausgangs-Clipping. Er prüft keine optische ReLU oder vollständige Sortierung.

Wichtige Grenze: Bei (255,0) clippt unser alter Vergleichskern 255/256 auf 127/256. Eine Min/Max-Rekonstruktion daraus ergäbe (128,127). Deshalb bleibt die Weiterleitung der Originalschlüssel in der bestehenden Evaluation erhalten.

Primärquellen, geprüft am 05.10.2026: [non_linear.rs](#), [backend_driver.rs](#), [API-Vertrag](#). Pinned Commit: 72a2d99. Der CPU-Ersatz cos ist keine kalibrierte optische tcos-Kennlinie.

Ergebnisse, Grenzen und nächste Schritte

5. Unveränderte Ergebnisse der Differenz-Basis

$N = 2, 4, \dots, 16$ bzw. 256; je 1.000 gespeicherte Arrays pro Größe und Eingabefamilie. Ziehen ohne Zurücklegen erzeugt verschiedene Schlüssel; Ziehen mit Zurücklegen erlaubt Duplikate. Beide Mappings verwenden dieselben 24.000 Eingaben, unabhängige Rauschströme und $\eta = \{0; 0,05; 0,10; 0,15; 0,20; 0,25\}$. Das ergibt 288.000 Sortierläufe.

Bitonic / Rang in %	$\eta \leq 0,10$	$\eta = 0,15$	$\eta = 0,20$	$\eta = 0,25$
4 Bit, $N=16$, verschieden	100 / 100	99,6 / 99,9	95,5 / 95,0	84,3 / 83,1
4 Bit, $N=16$, Duplikate erlaubt	100 / 100	99,6 / 99,7	95,1 / 95,2	87,9 / 80,2
8 Bit, $N=256$, verschieden	100 / 100	93,9 / 96,1	44,7 / 45,4	5,9 / 4,0
8 Bit, $N=256$, Duplikate erlaubt	100 / 100	95,5 / 94,7	54,8 / 38,3	10,4 / 3,1

Erfolg bedeutet: das gesamte Array ist aufsteigend sortiert und jeder Originaldatensatz genau einmal enthalten. Stabilität wird separat erfasst. Bei $\eta \leq 0,10$ waren alle getesteten Größen erfolgreich; 1.000/1.000 ist eine Beobachtung, keine universelle Garantie. Bei großen N können wenige Fehlvergleiche den gesamten Lauf scheitern lassen.

6. Grenzen und nächste Untersuchungen

Ungeklärt: tatsächliche Q.ANT-Kennlinie, ENOB, Rauschverteilung, physische Batchgrenzen, Datentransfers, Aufrufplatz und optische Kaskadierbarkeit. BF16 ist ein Schnittstellenformat. Die bisherigen Gaußwerte beschreiben keine gemessene Hardware.

Nächster Kernschritt: Eine Vergleichsfunktion aus der unterstützten periodischen Nichtlinearität herleiten und zunächst Gleichheit sowie alle kleinsten Schlüsselabstände prüfen. Zusätzliche Operationen, Transfers und Näherungsfehler erfassen. Ausgangsskala und Rauschen hierfür neu begründen; η der Differenzsimulation nicht ungeprüft übertragen.

Danach beide Sortierer bei derselben Korrektheitsanforderung vergleichen und Q.ANT-Messungen zu Kennlinie, Rauschen und Zeitgrenzen anstreben. Regenerationsintervalle erst bei belegtem Kaskadenpfad variieren. Energie folgt aus belastbaren Zeit- und Leistungsdaten. PRISM bleibt verwandte Arbeit zu photonischem Scoring mit elektronischer Top-k-Auswahl.

7. Projekt und Ergebnisdateien

<https://github.com/Bharadwaj-Srikumar/qant-sorting-pilot>

Einstieg: README.md, comparison.py und sorting_schedules.py erklären Vergleich und Sortierabläufe. run_noise_sweep.py reproduziert den Sweep; results/reference/accuracy.csv enthält alle 288 Konfigurationen. SDK-Ergebnisse: results/sdk_reference/results.csv, Filter mode=sdk_cpu.

Neu im Projekt-ZIP: docs/QANT_NONLINEARITY.md enthält Quellenbefund und Grenzen. check_sdk_nonlinearity.py reproduziert die separate CPU-Prüfung; validation/nonlinearity_check.json enthält deren Ergebnis. Die GitHub-Aktualisierung ist durch einen aktuellen Zugriffsfehler blockiert. Basis ist der letzte verifizierte Commit 22ef167.

Erbetenes Feedback: Ist dieser eingegrenzte Vergleich mit der Untersuchung der tatsächlich verfügbaren Nichtlinearität als Grundlage für die Anmeldung der Masterarbeit geeignet?